Yahye F

Assignment 6.
Trees & Heaps

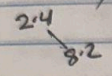
* smaller goes left
bigger goes right

1. (text) Type of tree [9 points].

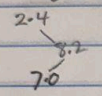
9. [2.4, 8.2, 7.0, 5.2, 1.0, 6.5, 2.6, 3.6, 7.9, 0.5, 6.6, 9.9, 1.2]

① Insert 2.4
It becomes the root.

② Insert 8.2, 8.2 > 2.4 so it goes right.



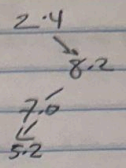
③ Insert 7.0
7.0 > 2.4, → R to 8.2
7.0 < 8.2 → left.



Skipped to steps six to 12

Step ⑫ Insert 9.9
9.9 > 2.4 → 8.2
9.9 > 8.2 → right

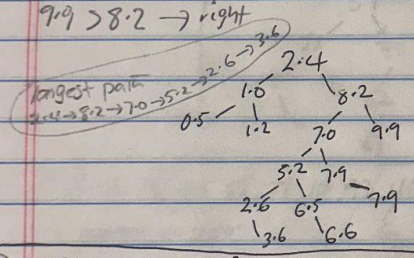
④ Insert 5.2
5.2 > 2.4, → R to 8.2
5.2 < 8.2, go L to 7.0
5.2 < 7.0 go left.



⑤ add 1.0
since 1.0 < 2.4, it goes left.

⑥ Insert 6.5
6.5 > 2.4 → right to 8.2
6.5 < 8.2 → Left to 5.2
6.5 7.0 → left to 5.2
6.5 > 5.2, so it goes right.

Step ⑬ Insert 1.2
1.2 < 2.4 → 1.0
1.2 > 1.0 → goes right.



height = 5

(b) ["Platinum", "Collander", "Sour", "Willow", "Rasx", "Gingerbread", "Hiuon"]

Step 1 "Platinum" → become root.

Step 2 "Coliander"
"C" comes before "P", so left.

Step 3 "Sour"
"S" comes after "P", so Right.

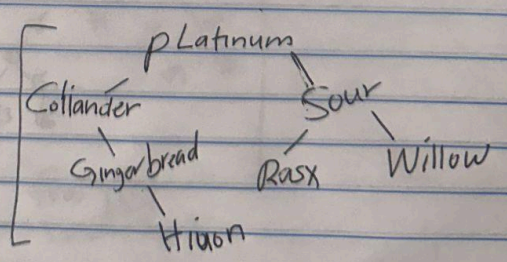
Step 4 "Willow"
"W" > "P", so right to sour.
"W" > "S" so R.

Step 5 "Rasx"
"R" > "P" so Right to sour.
"R" < "S" so Left.

Step 6 "Gingerbread"
"G" < "P" → L to Coliander
"G" > "C" so right.

Step 7 "Hiuon"
"H" < "P" → L to Coliander
"H" > "C" → R to Ginger bread
"H" > "G" so it goes Right.

h=3



R<C<L

1(c) [1, 37, 11, 71, 62, 4, 61, 1, 11, 4] add it in order *

Step 1 1 Root.

step2 37, 37>1, → right

11, 11>1 → right → 37
11<37 → left

71, 71>1 → 37
71>37 → R

62. 62>1 → 37
62>37 → 71
62<71 → Left.

4, 4>1 → 37
4<37 → 11
4<11 → L

61, 61>1 → 37
61>37 → 71
61<71 → 62
61<62 → left.

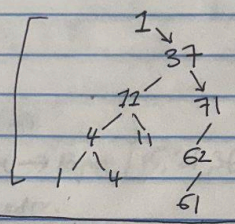
Add 1,
1=1, so I go right
at 37, 1<37 → L
11, 1<11 → L
at 4, 1<4 → Left.

Insert 11
11>1 → 37
11<37 → go to 11
they are the same so I go to the right.

Insert 4
4>1 → 37
4<37 → 11
4<11 → 4
equal, then go Right.

Final BST.

height = 4



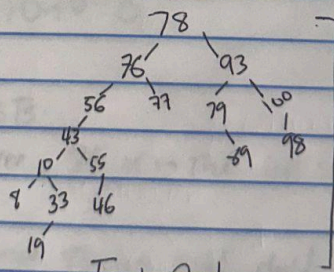
d) [78, 76, 56, 43, 55, 77, 10, 93, 46, 79, 33, 8, 89, 100, 98, 19]

I am inserting in order.

1) 78 → This the root.
2) 76, since 76<78, left
3) 56, 56<78 → 76
56<76 → left.
4) 43, so for 43<78 → 76
43<76 → 56
43<56 → left.
5) 55, 55<78 → 76
55<76 → 56
55<56 → 43
55>43 → Right.
6) 77, 77<78 → 76
77>76 → right
7) 10, so for the 10<78 → 76
10<76 → 56
10<56 → 43
10<43 → so it goes left of 43.
8) 93 > 78 → right
9) 46, 46<78 → 76
46<76 → 56
46<56 → 43
46>43 → 55
46<55 → Left.

step 10 Inserting 79
So 79>78 → 93
79<93 → left.

Step 10 — 16
|||||



h=6

Final BST

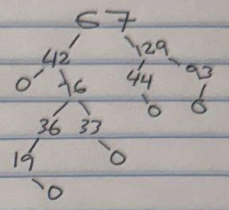
2. (text). Bst traversal.

Given rule: node.data = $\frac{\text{left.data} + \text{right.data}}{2}$.
0 for empty nodes and round up.

a) Preorder traversal. root → left → Right.

order visited ⇒ [68, 21, 15, 54, 46, 36, 37, 59, 65, 92, 80, 87, 97, 93]

∴ 68 → $21 + \frac{92}{2} = 67$
21 → $15 + \frac{54}{2} = 42$
15 → 0
54 → $46 + \frac{59}{2} = 76$
46 → $36 + \frac{0}{2} = 36$
36 → $0 + \frac{37}{2} = 19$
37 → 0
59 → $0 + \frac{65}{2} = 33$
65 → 0
92 → $80 + \frac{97}{2} = 129$
80 → $0 + \frac{87}{2} = 44$
87 → 0
97 → $93 + 0 = 93$
93 → 0



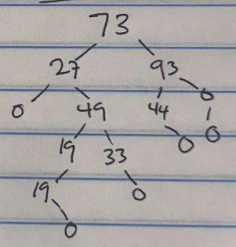
a) Inorder traversal. [ ]

Left → Root → Right [15, 21, 36, 37, 46, 54, 59, 65, 68, 80, 87, 92, 93, 97]

New Value in order.

15 → 0
$21 \to 0 + \frac{54}{2} = 27$
$36 \to 0 + \frac{37}{2} = 19$
37 → 0
46 → 19 + 0 = 19
$54 \to 19 + \frac{59}{2} = 49$
$59 \to 0 + \frac{65}{2} = 33$
65 → 0
$68 \to 27 + \frac{92}{2} = 73$
$80 \to 0 + \frac{87}{2} = 44$
87 → 0
$92 \to 44 + \frac{97}{2} = 93$
93 → 0
97 → 0 + 0 = 0

Resulting tree.



B. NO, both are not BSTs.
because in preorder tree, 76 is in the left subtree of 67 so it breaks Bst rule.
also in Inorder tree node 93 has right child 0, and that also breaks Bst rule

C) Are resulting tree AVLs?
both of the new tree are not AVLs because AVL must be a Bst first and since these tree are not Bst, they cannot be AVLs.

Question 7.

4) findElement

The algorithm has a time complexity of O(n) and a space complexity of O(h), where n is the number of positions in the array and h is the height of the tree. This is because the algorithm may need to visit every valid node in the binary tree one time in the worst case while rebuilding the original values from the tree's structure. At each node, it only does a small constant amount of work, such as checking whether the node is empty, comparing the current value to the target, and calculating the left and right child values. Because of that, the total work grows linearly with the number of nodes, so the time complexity is O(n). The space complexity is O(h) because the recursive calls are stored on the call stack, and the deepest the recursion can go is the height of the tree.

5) It has a time complexity of O(n) and a space complexity of O(k), where n is the number of elements in the heap and k is the number of strings whose length is less than x. This is because the method iterates through the entire heap array once, checking each string to see if its length is less than x. Since each check takes constant time, performing it n times results in linear time complexity. The space used grows with the number of matching strings stored in the result list, which is k.

6) The overall algorithm has a time complexity of O(n + k log n) and a space complexity of O(k), where n is the number of nodes in the tree and k is the number of nodes in the range. This is because the algorithm first traverses the tree to collect all values in the range, which takes O(n) time. Then, it deletes each of the k values, and each deletion takes O(log n) time since the tree is balanced. The space complexity is O(k) because the algorithm stores the values to be deleted in a list before removing them.